FATEC IPIRANGA

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS (ADS)

ESTRUTURA DE DADOS
Atividade N1-2: Representação Estado Memória

Prof. Veríssimo
Auno: Yan Kevin de Sousa Costa

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O computador reserva um espaço na memória para a função main. Esse espaço ainda está vazio, pois nenhuma variável foi criada.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

Executa a criação da variável. Na memória, dentro do espaço da main, surge a variável **valor** e o número **12345** é gravado dentro dela. Agora, esse dado está disponível para ser usado.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O programa lê o número **12345** que está guardado na memória e o coloca dentro da frase que será exibida. O texto "Invertendo o numero 12345: " é enviado para a tela

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O programa interrompe a execução da main e se prepara para ir até a linha 4. Ele envia uma cópia do número **12345** para ser usado pela função

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O programa executou o "salto" da linha 18 e agora está no cabeçalho da função inverterNumero

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O programa executa a linha 5, a condição para checar ser o valor de n é exatamente igual a zero. Como o resultado é falso, o bloco de código que está entre as chaves do if (linhas 6 e 7) é ignorado. O processador então se prepara para pular diretamente para a linha 9, onde a estrutura while será iniciada.

```c
#include <stdio.h>

void inverterNumero(int n) {
if (n == 0) {
printf("0");
return;
}
while (n > 0) {
printf("%d", n % 10);

n = n / 10;
}
}
int main() {
int valor = 12345;
printf("Invertendo o numero %d: ", valor);
inverterNumero(valor);
printf("\n");
return 0;
}
```

O computador faz uma comparação consultando o quadro de memória da função inverterNumero, onde lê que o valor atual de n 12345. O programa está apenas validando a condição para decidir se deve processar o próximo dígito ou encerrar a tarefa.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O processador acessa o valor de n na memória e calcula 12345 (mod 10). Esse resultado é enviado, fazendo com que o número 5 apareça logo após o texto que já estava na tela . Neste momento, a memória da função ainda mantém o valor original de n , pois o cálculo do resto da divisão apenas utiliza o valor sem alterá-lo. O objetivo é garantir que o dígito mais à direita seja o primeiro a ser impresso, para a lógica de inversão.

```c

#include <stdio.h>

void inverterNumero(int n) {
if (n == 0) {
printf("0");
return;
}
while (n > 0) {
printf("%d", n % 10);

n = n / 10;
}
}
int main() {
int valor = 12345;
printf("Invertendo o numero %d: ", valor);
inverterNumero(valor);
printf("\n");
return 0;
}
```

O computador pega o valor atual de n (12345), divide por 10 e armazena apenas o resultado inteiro, que é 1234

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

A execução retorna para a linha 9 para reavaliar a condição do laço while. O computador consulta novamente o quadro de memória da função inverterNumero e lê o valor atualizado de n, que agora é 1234.

Known limitations

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O processador acessa o valor de n na memória e calcula 1234 (mod 10). Esse resultado é enviado, fazendo com que o número 4 apareça logo após o texto que já estava na tela . Neste momento, a memória da função ainda mantém o valor original de n , pois o cálculo do resto da divisão apenas utiliza o valor sem alterá-lo. O objetivo é garantir que o dígito mais à direita seja o primeiro a ser impresso, para a lógica de inversão.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O computador pega o valor atual de n (1234), divide por 10 e armazena apenas o resultado inteiro, que é 123

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

A execução retorna para a linha 9 para reavaliar a condição do laço while. O computador consulta novamente o quadro de memória da função inverterNumero e lê o valor atualizado de n, que agora é 123.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O processador acessa o valor de n na memória e calcula 123 (mod 10). Esse resultado é enviado, fazendo com que o número 3 apareça logo após o texto que já estava na tela . Neste momento, a memória da função ainda mantém o valor original de n , pois o cálculo do resto da divisão apenas utiliza o valor sem alterá-lo. O objetivo é garantir que o dígito mais à direita seja o primeiro a ser impresso, para a lógica de inversão.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O computador pega o valor atual de n (123), divide por 10 e armazena apenas o resultado inteiro, que é 12

known limitations

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

A execução retorna para a linha 9 para reavaliar a condição do laço while. O computador consulta novamente o quadro de memória da função inverterNumero e lê o valor atualizado de n, que agora é 12.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O processador acessa o valor de n na memória e calcula 12 (mod 10). Esse resultado é enviado, fazendo com que o número 2 apareça logo após o texto que já estava na tela . Neste momento, a memória da função ainda mantém o valor original de n , pois o cálculo do resto da divisão apenas utiliza o valor sem alterá-lo. O objetivo é garantir que o dígito mais à direita seja o primeiro a ser impresso, para a lógica de inversão.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

O computador pega o valor atual de n (12), divide por 10 e armazena apenas o resultado inteiro, que é 1

```
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12 n = n / 10;
13 }
14 }
15 int main() {
16 int valor = 12345;
17 printf("Invertendo o numero %d: ", valor);
18 inverterNumero(valor);
19 printf("\n");
20 return 0;
21 }
```

A execução retorna para a linha 9 para reavaliar a condição do laço while. O computador consulta novamente o quadro de memória da função inverterNumero e lê o valor atualizado de n, que agora é 1.

```c

#include <stdio.h>

void inverterNumero(int n) {
if (n == 0) {
printf("0");
return;
}
while (n > 0) {
printf("%d", n % 10);

n = n / 10;
}
}
int main() {
int valor = 12345;
printf("Invertendo o numero %d: ", valor);
inverterNumero(valor);
printf("\n");
return 0;
}
```

O processador realiza o cálculo de resto da divisão por 10, como 1 é menor que 10, o resultado é o próprio 1, esse valor é enviado para a tela, terminando a sequência numérica.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12
13
14 n = n / 10;
15 }
16 }
17 int main() {
18 int valor = 12345;
19 printf("Invertendo o numero %d: ", valor);
20 inverterNumero(valor);
21 printf("\n");
```

O programa faz a divisão. O processador realiza uma divisão inteira do valor atual de n (1) por 10. O resultado da operação 1 / 10 é 0. Esse novo valor (0) é gravado na memória, substituindo o valor anterior

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12
13
14 n = n / 10;
15 }
16 }
17 int main() {
18 int valor = 12345;
19 printf("Invertendo o numero %d: ", valor);
20 inverterNumero(valor);
21 printf("\n");
```

a execução volra para validar a condição do laço while (n > 0). O computador consulta o quadro de memória da função e lê o valor atual de n, que agora é 0. Fazendo a comparação, verifica que a condição "0 é maior que 0" é falsa. Como a exigência para continuar no laço não é mais atendida, o fluxo de execução é desviado para fora do bloco de repetição, saltando para a linha 14, que marca o fechamento da função inverterNumero.

```
1
2  #include <stdio.h>
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12
13
14 n = n / 10;
15 }
16 }
17 int main() {
18 int valor = 12345;
19 printf("Invertendo o numero %d: ", valor);
20 inverterNumero(valor);
21 printf("\n");
```

O programa atinge a chave de fechamento da função inverterNumero, o que sinaliza ao computador que a tarefa daquela função foi concluída, voltando para a função main.

Known limitations

```c
#include <stdio.h>

void inverterNumero(int n) {
if (n == 0) {
printf("0");
return;
}
while (n > 0) {
printf("%d", n % 10);



n = n / 10;
}
}
int main() {
int valor = 12345;
printf("Invertendo o numero %d: ", valor);
inverterNumero(valor);
printf("\n");
```

Após ter impresso todos os números (54321), o cursor do teclado estava parado logo após o número 1. Esse comando move o cursor para o início da linha de baixo. Organizando o resultado, garantindo que a resposta do programa não fique misturada.

```
3
4  void inverterNumero(int n) {
5  if (n == 0) {
6  printf("0");
7  return;
8  }
9  while (n > 0) {
10 printf("%d", n % 10);
11
12
13
14 n = n / 10;
15 }
16 }
17 int main() {
18 int valor = 12345;
19 printf("Invertendo o numero %d: ", valor);
20 inverterNumero(valor);
21 printf("\n");
22 return 0;
23 }
```

O comando return 0; indica que a função principal (main) terminou com sucesso, sem nenhum erro durante a execução.